

C++ ile GLFW

Bilgisayar Grafiđi, Matematik ve OpenGL Rehberi

Teoriden Pencereye, Matematikten İlk Üçgene Kapsamlı Başvuru

Bu belge önce bilgisayar grafiđinin **nasıl çalıştığını** (grafik boru hattı, GPU, rasterizasyon), sonra grafiđin **matematikini** (vektörler, matrisler, dönüşümler, projeksiyon) ve ardından **GLFW + OpenGL** ile adım adım kod yazmayı **çok sayıda çalışan örnekle** anlatır. Tüm kod örnekleri İngilizce isimlendirmeye, üretim projelerindeki gerçek stile uygun yazılmıştır.

Kapsam — Kısım 1 (Grafik Temelleri): bilgisayar grafiđi nedir • raster vs vektör • GPU & CPU • grafik boru hattı • koordinat uzayları • renk & framebuffer • çift tamponlama.

Kapsam — Kısım 2 (Grafik Matematigi): vektörler • nokta & çapraz çarpım • matrisler & sayısal örnekler • öteleme/ölçek/döndürme • homojen koordinatlar • Model-View-Projection • kamera (LookAt) • perspektif & ortografik projeksiyon.

Kapsam — Kısım 3 (GLFW): kurulum & CMake • ilk pencere • hata callback'i • olay döngüsü • GLAD • klavye/fare/scroll girdisi • tam ekran & monitör • delta time & FPS sayacı.

Kapsam — Kısım 4 (OpenGL Çizim): GLSL derinlemesine • VBO/VAO/EBO • tam ilk üçgen programı • uniform'lar • dokular & çoklu doku karışımı • glm ile dönüşümler • 3B küp & derinlik testi • FPS kamera sınıfı • Phong aydınlatma.

Kapsam — Kısım 5 (Projeler): soyutlanmış Shader sınıfı • hata denetimi yardımcıları • dönen renkli küp (tam program) • hata ayıklama tablosu • sonraki adımlar.

Hazırlanma Tarihi: 14 Temmuz 2026

C++17 • GLFW 3.x • OpenGL 3.3 Core • GLAD • GLM

Contents

1	Bilgisayar Grafiği Nedir?	3
1.1	Raster ve Vektör Grafik	3
1.2	Gerçek Zamanlı ve Offline Grafik	3
1.3	CPU ve GPU: İş Bölümü	4
2	Grafik Boru Hattı (Graphics Pipeline)	4
2.1	Boru Hattının Aşamaları	4
2.2	Bir Üçgenin Yolculuğu	5
3	Koordinat Sistemleri ve Uzaylar	5
3.1	Normalized Device Coordinates (NDC)	5
4	Renk, Framebuffer ve Çift Tamponlama	6
4.1	Renklerin Temsili	6
4.2	Framebuffer ve Çift Tamponlama	6
5	Trigonometri: Grafiğin Açık Dili	8
5.1	Radyan ve Derece	8
5.2	Birim Çember: Sinüs ve Kosinüs	9
5.3	Temel Özdeşlikler	9
5.4	Tanjant ve atan2: Noktadan Açığa	10
5.5	Trigonometri Neden Döndürme Matrisinin İçinde?	10
6	Vektörler	11
6.1	Temel İşlemler	11
6.2	Nokta Çarpım (Dot Product)	12
6.3	Çapraz Çarpım (Cross Product)	12
7	Matrisler ve Dönüşümler	12
7.1	Matris Çarpımı ve Birim Matris	13
7.2	Öteleme (Translation)	13
7.3	Ölçekleme (Scale)	13
7.4	Döndürme (Rotation)	14
7.5	Dönüşümleri Birleştirme	14
8	Homojen Koordinatlar	15
8.1	Perspektif Bölme	15
9	View ve Projection: Kamera Matematiği	15
9.1	View Matrisi (LookAt)	15
9.2	Projection Matrisi	16
9.3	Hepsini Birleştirmek	16
10	Kurulum ve Derleme	18
10.1	Linux'ta Paketlerin Kurulumu	18
10.2	CMake ile Proje Yapısı	18
11	İlk Pencere	19
11.1	Ana Döngünün Anatomisi	21

12 Callback'ler ve Olay İşleme	21
12.1 Pencere Boyutlandırma	21
12.2 Klavye Girdisi: İki Yöntem	21
12.3 Fare Girdisi	22
13 Zaman, Delta Time ve FPS	23
13.1 Basit FPS Sayacı	23
14 Shader'lar ve GLSL Dili Derinlemesine	25
14.1 Shader Türleri ve Boru Hattındaki Yeri	25
14.2 Bir Shader'ın İskeleti	25
14.3 Veri Tipleri	26
14.4 Swizzling: Vektör Bileşenlerine Erişim	26
14.5 Giriş, Çıkış ve Nitelikler (in / out / uniform)	26
14.6 Yerleşik Değişkenler ve Fonksiyonlar	27
15 VBO, VAO ve İlk Üçgen	28
15.1 Shader'ları Yazmak	28
15.2 Shader Derleme ve Bağlama	29
15.3 Köşe Verisi, VAO/VBO Kurulumu ve Çizim	29
15.4 Tam, Kendi Kendine Yeten İlk Üçgen Programı	30
16 Öznitelikler ve Renk İnterpolasyonu	31
17 EBO ile Dikdörtgen ve Uniform'lar	32
17.1 EBO (Element Buffer Object)	32
17.2 Uniform'lar: CPU'dan Shader'a Veri	33
18 Dokular (Textures)	34
18.1 İki Dokuyu Karıştırmak (Texture Units)	35
19 glm ile Dönüşümleri Uygulamak	35
20 3B'ye Geçiş: Küp ve Derinlik Testi	36
20.1 Derinlik Testini Açmak	36
20.2 MVP ile Dönen Küp	36
21 Kamera Sınıfı (FPS Kamera)	37
22 Temel Aydınlatma: Phong Modeli	39
23 Soyutlanmış Shader Sınıfı	41
24 Tam Program: Dönen Renkli Küp	43
25 Kaynak Yönetimi ve Sık Yapılan Hatalar	45
25.1 Kaynakları Temizlemek	45
25.2 Hata Ayıklama Kontrol Listesi	45
25.3 Nereden Devam Etmeli?	45

KISIM 1

Bilgisayar Grafiğinin Temelleri

1. Bilgisayar Grafiği Nedir?

Bilgisayar grafiği, sayısal verileri (noktalar, renkler, geometri) ekrandaki piksellere dönüştürme bilimidir. Bir üçgenin köşe koordinatlarından yola çıkıp, onu ekranda renkli bir yüzey olarak görmemizi sağlayan tüm matematiksel ve donanımsal süreç bu başlık altındadır. GLFW ve OpenGL öğrenmeden önce *neyi* kontrol ettiğimizi anlamak, sonraki her satırı çok daha anlamlı kılar.

1.1 Raster ve Vektör Grafik

İki temel grafik temsili vardır. Aradaki farkı bilmek, GPU'nun neden piksellerle çalıştığını anlamının anahtarıdır.

Tür	Nasıl saklanır?	Örnek
Raster	Piksellerden oluşan bir ızgara; her piksel bir renk değeri tutar. Büyütünce bozulur (pikselleşme).	PNG, JPEG, ekran görüntüsü, fotoğraf.
Vektör	Matematiksel şekiller (doğru, eğri, çokgen). Sonsuz büyütülebilir, bozulmaz.	SVG, font, CAD çizimi.

Ekranımız bir raster aygıttır: sonunda her şey piksele dönmek zorundadır. OpenGL ile aslında *vektörel* tanımlar (köşe koordinatları) veririz; GPU bunları **rasterizasyon** adı verilen adımda piksellere çevirir.

1.2 Gerçek Zamanlı ve Offline Grafik

Yaklaşım	Açıklama
Gerçek zamanlı (real-time)	Kareyi milisaniyeler içinde üretmek gerekir (60 FPS $\approx$ 16.6 ms/kare, 144 FPS $\approx$ 6.9 ms/kare). Oyunlar, simülasyonlar. Rasterizasyon tabanlıdır ve OpenGL/GLFW bu dünyadadır .
Offline (ışın izleme)	Kare başına saniyeler/dakikalar harcanabilir. Fotogerçekçilik hedeflenir. Sinema efektleri (ray tracing / path tracing).

Bilgi

Bu rehberdeki her şey **gerçek zamanlı rasterizasyon** üzerinedir. Amacımız kareyi mümkün olan en hızlı şekilde çizip ekrana basmaktır; bunun için GPU'nun paralel gücünü kullanırız. 60 FPS demek, tüm sahneyi (girdi, mantık, çizim) 16.6 milisaniyede bitirmek zorunda olmak demektir — bu yüzden her çağrının maliyeti önemlidir.

1.3 CPU ve GPU: İş Bölümü

CPU birkaç güçlü çekirdekle sıralı işleri iyi yapar; GPU ise binlerce basit çekirdekle *aynı işlemi* milyonlarca veri üzerinde paralel yürütür. Bir ekran $1920 \times 1080 = 2$ milyon piksel içerir; her karede hepsini boyamak paralellik ister. İşte GPU tam bu iş için vardır.

CPU	GPU
Az sayıda (4–32) güçlü çekirdek	Binlerce basit çekirdek
Karmaşık, dallı, sıralı mantık	Basit, paralel, aynı-işlem
Düşük gecikme (latency)	Yüksek verim (throughput)

Programlama modeli

CPU'da C++ ile *host* kodu yazarız (GLFW penceresi, oyun mantığı). GPU'da ise GLSL ile *shader* yazarız (her köşe/piksel için çalışan minik programlar). OpenGL bu ikisi arasındaki köprüdür: CPU'daki veriyi GPU'ya yükler ve shader'ları çalıştırır. Verinin CPU'dan GPU'ya kopyalanması pahalıdır; bu yüzden veriyi bir kez yükleyip defalarca kullanmayı hedefleriz.

2. Grafik Boru Hattı (Graphics Pipeline)

Grafik boru hattı, köşe verisinin (vertex) ekrandaki renkli piksele dönüşene kadar geçtiği aşamalar zinciridir. OpenGL 3.3 Core'da bu boru hattının bazı aşamalarını biz programlarımız (shader), bazıları ise sabit-fonksiyondur (GPU donanımı yapar). Tüm OpenGL kodunuz aslında bu boru hattını beslemekten ibarettir.

2.1 Boru Hattının Aşamaları

Aşama	Ne yapar?	Kim?
1. Vertex verisi	VBO'dan köşe özniteliklerini (konum, renk, UV) okur.	Biz (VAO)
2. Vertex Shader	Her köşeyi işler; genelde MVP ile clip uzayına taşır.	Biz (GLSL)
3. Primitive Assembly	Köşeleri üçgen/çizgi/nokta olarak gruplar.	GPU
4. (Geometry Shader)	Opsiyonel: yeni geometri üretebilir.	Biz (ops.)
5. Rasterizasyon	Üçgeni kapsadığı piksellere (fragment) böler; öznitelikleri interpolasyonla yayar.	GPU
6. Fragment Shader	Her fragment'in nihai rengini hesaplar (doku, ışık).	Biz (GLSL)
7. Testler & Karıştırma	Derinlik testi, stencil, alpha blend uygulanır.	GPU
8. Framebuffer	Sonuç piksel renk tamponuna yazılır.	GPU

Bilgi

OpenGL 3.3 Core Profile'da **en az iki shader yazmak zorunludur**: bir *vertex shader* ve bir *fragment shader*. Bunlar olmadan hiçbir şey çizilmez. Rehberin 4. kısmında bunları satır satır yazacağız.

2.2 Bir Üçgenin Yolculuğu

Somut düşünelim: ekranın ortasına bir üçgen çizmek istiyoruz. Üç köşesinin konumunu C++ dizisinde tanımlarız → VBO ile GPU belleğine yükleriz → vertex shader her köşeyi `gl_Position` olarak clip uzayına yerleştirir → rasterizasyon üçgenin içindeki tüm pikselleri bulur → fragment shader her pikseli (ör. turuncuya) boyar → sonuç ekrana gelir.

Zihinsel model

OpenGL bir "durum makinesidir" (state machine). Sen "şu tamponu bağla", "şu shader'ı kullan", "şimdi çiz" dersin; OpenGL o anki duruma göre boru hattını çalıştırır. Bu yüzden çağrı *sırası* çok önemlidir. Bir nesneyi *bind* etmek, "sonraki komutlar bu nesneyi etkilesin" demektir.

3. Koordinat Sistemleri ve Uzaylar

Bir 3B nesne ekrana gelene kadar birçok koordinat uzayından geçer. Bu dönüşümlerin tamamı matris çarpımlarıdır (Kısım 2). Şimdilik hangi uzayların olduğunu ve ne işe yaradığını kavrayalım.

Uzay	Açıklama	Dönüşüm
Local (Model)	Nesnenin kendi merkezine göre koordinatları.	—
World	Nesnenin sahnedeki (dünyadaki) yeri.	Model matrisi
View (Kamera)	Her şeyin kameraya göre konumu.	View matrisi
Clip	Perspektif uygulanmış, kırpma yapılan uzay.	Projection matrisi
NDC	Normalized Device Coords: $[-1, 1]^3$ küpü.	Perspektif bölme (w)
Screen	Piksel koordinatları (viewport).	Viewport dönüşümü

MVP zinciri

Bir köşenin ekrandaki yeri şu çarpımla bulunur:

$$\mathbf{v}_{clip} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \cdot \mathbf{v}_{local}$$

Bu üç matrisi (Model, View, Projection) vertex shader'a uniform olarak göndeririz. Kısım 2 bu matrislerin nasıl kurulduğunu, Kısım 4 ise koda nasıl döküldüğünü anlatır.

3.1 Normalized Device Coordinates (NDC)

Vertex shader'dan çıkan `gl_Position` clip uzayındadır. GPU bunu w bileşenine bölerek NDC'ye geçer. NDC, her eksen $[-1, 1]$ arasında olan bir küptür: sol alt köşe $(-1, -1)$, sağ üst $(+1, +1)$,

merkez (0,0). Bu küpün dışında kalan her şey **kırpılır** (clip). En basit üçgeni doğrudan NDC’de tanımlayarak (matris kullanmadan) çizebiliriz — ilk örneğimizde tam olarak bunu yapacağız.

NDC Düzlemi (z’yi yok sayarsak)	
(-1, +1) sol üst	(+1, +1) sağ üst
merkez (0, 0)	
(-1, -1) sol alt	(+1, -1) sağ alt

Dikkat

OpenGL’de NDC’nin Y eksenini **yukarı** doğrudur (matematiksel gelenek), ama ekran piksel koordinatlarında Y genelde **aşağı** doğrudur. Doku (texture) koordinatlarında bu ters çevrim sık sık kafa karıştırır; ilerde göreceğiz.

4. Renk, Framebuffer ve Çift Tamponlama

4.1 Renklerin Temsili

Ekranda renk, kırmızı-yeşil-mavi (RGB) ışığın toplamıyla oluşur (toplamsal renk karışımı). OpenGL’de renkleri genelde [0.0, 1.0] aralığında float olarak veririz; bazen alfa (saydamlık) ile birlikte RGBA olarak.

R	G	B	A	Renk
1.0	0.0	0.0	1.0	Kırmızı
0.0	1.0	0.0	1.0	Yeşil
0.0	0.0	1.0	1.0	Mavi
1.0	1.0	1.0	1.0	Beyaz
0.2	0.3	0.3	1.0	Koyu turkuaz (klasik arka plan)

4.2 Framebuffer ve Çift Tamponlama

Framebuffer, o an ekranda gösterilecek pikselleri tutan bellek bloğudur. Eğer tek bir tampona hem çizip hem gösterirsek, yarı çizilmiş kareler görünür ve görüntü titrer (flicker, tearing). Çözüm **çift tamponlamadır**:

Tampon	Rol
Back Buffer	GPU bir sonraki kareyi buraya <i>gizlice</i> çizer.
Front Buffer	Ekranda o an görünen kare.

Kare bittiğinde iki tampon **yer değiştirir** (swap). GLFW’de bunu `glfwSwapBuffers(window)` çağırısı yapar. Böylece kullanıcı hiçbir zaman yarı çizilmiş kare görmez.

VSync

Tamponların yer değiştirmesini ekranın yenileme hızıyla (60/144 Hz) senkronlamak "VSync"tir. GLFW'de `glfwSwapInterval(1)` ile açılır; ekran yırtılmasını (tearing) engeller ama FPS'i yenileme hızına sabitler. `glfwSwapInterval(0)` ile kapatılır (FPS sınırsız, ölçüm için faydalı).

İpucu

Kısım 1'i bitirdin! Artık grafiğin *ne* yaptığını biliyorsun: vektörel geometriyi GPU boru hattından geçirip, koordinat uzayları arasında dönüştürerek, çift tamponlu bir framebuffer'a rasterize etmek. Şimdi bu dönüşümlerin *matematiğine* geçiyoruz.

KISIM 2

Grafik Matematiği

Bu kısım grafiğin dilini kurar. Endişelenme: her kavramı hem matematiksel gösterimle hem de birazdan kullanacağımız **GLM** kütüphanesindeki karşılığıyla vereceğiz. GLM (OpenGL Mathematics), GLSL sözdizimini C++'a taşıyan başlık-only bir kütüphanedir; `glm::vec3`, `glm::mat4` gibi tipleri sağlar. Bütün kod örneklerinde İngilizce isimler kullanacağız.

5. Trigonometri: Grafiğin Açık Dili

Trigonometri, açılar ile uzunluklar arasındaki ilişkidir ve grafiğin her yerinde karşımıza çıkar: döndürme matrisleri, dairesel hareket, kamera yönü, dalga efektleri, aydınlatma... Döndürmeyi gerçekten anlamak için önce sinüs ve kosinüsü sağlam oturtmak gerekir. Bu bölüm sıfırdan kurar.

5.1 Radyan ve Derece

Açıyı iki birimle ölçeriz. **Derece** günlük birimdir (tam tur = 360°). **Radyan** ise matematiğin ve tüm grafik kütüphanelerinin doğal birimidir: bir açının radyan değeri, birim çemberde o açının gerdiği yay uzunluğudur. Tam tur = 2π radyandır.

$$180^\circ = \pi \text{ radyan} \quad \Rightarrow \quad \text{radyan} = \text{derece} \times \frac{\pi}{180}$$

Derece	Radyan	sin	cos
0°	0	0	1
30°	$\pi/6$	0.5	≈ 0.866
45°	$\pi/4$	≈ 0.707	≈ 0.707
60°	$\pi/3$	≈ 0.866	0.5
90°	$\pi/2$	1	0
180°	π	0	-1
270°	$3\pi/2$	-1	0

Dikkat

C++ `<cmath>` fonksiyonları (`sin`, `cos`, `tan`) ve GLM'nin trigonometrik/döndürme fonksiyonları **radyan** bekler. Derece ile çalışmak istersen daima `glm::radians(derece)` ile çevir. Bu, yeni başlayanların bir numaralı hatasıdır — `sin(90)`, sandığın 1'i değil, 90 radyanın sinüsü olan ≈ 0.894 'ü verir.

```

1 #include <cmath>
2 #include <glm/glm.hpp>
3
4 const float PI = 3.14159265358979f;
5
6 float radians = glm::radians(90.0f); // 1.5708 (pi/2)
7 float degrees = glm::degrees(PI); // 180.0
8 float s = std::sin(radians); // 1.0
9 float c = std::cos(radians); // ~0.0

```

5.2 Birim Çember: Sinüs ve Kosinüs

En sağlam sezgi *birim çemberdir*: merkezi orijinde, yarıçapı 1 olan çember. Merkezden θ açısıyla çizilen yarıçapın çember üzerindeki ucu tam olarak $(\cos \theta, \sin \theta)$ noktasıdır.

çember üzerindeki nokta = $(\cos \theta, \sin \theta)$

Yani: **kosinüs** noktanın X (yatay) bileşeni, **sinüs** ise Y (dikey) bileşenidir. θ arttıkça nokta çember boyunca saat yönünün tersine döner. Bu tek cümle, döndürme ve dairesel hareketin tamamının temelidir.

Açı θ	$\cos \theta$ (X)	$\sin \theta$ (Y)
0° (sağ)	1	0
90° (yukarı)	0	1
180° (sol)	-1	0
270° (aşağı)	0	-1

```

1 // Move an object along a circle of radius r (classic orbit)
2 float angle = (float)glfwGetTime(); // grows over time (radians)
3 float radius = 2.0f;
4 float x = radius * std::cos(angle); // horizontal position
5 float y = radius * std::sin(angle); // vertical position
6 glm::vec3 orbitPos(x, y, 0.0f); // point orbiting the origin

```

Bilgi

Dik üçgen tanımıyla köprü: bir dik üçgende $\sin \theta = \frac{\text{karşı}}{\text{hipotenüs}}$, $\cos \theta = \frac{\text{komşu}}{\text{hipotenüs}}$, $\tan \theta = \frac{\sin \theta}{\cos \theta} = \frac{\text{karşı}}{\text{komşu}}$. Birim çemberde hipotenüs = 1 olduğu için karşı kenar doğrudan sin, komşu kenar doğrudan cos olur — iki tanım aynı şeydir.

5.3 Temel Özdeşlikler

Grafik kodunda sık işine yarayacak birkaç özdeşlik:

Özdeşlik	Nerede işe yarar?
$\sin^2 \theta + \cos^2 \theta = 1$	Pisagor; birim vektör olduğunu doğrular, normalize kısayolu.
$\sin(-\theta) = -\sin \theta$	Tek fonksiyon (simetri).
$\cos(-\theta) = \cos \theta$	Çift fonksiyon (simetri).
$\sin(\theta) = \cos(\theta - 90^\circ)$	Sinüs, kosinüsün kaydırılmışıdır (faz farkı).
Periyot = 2π	Her ikisi de 2π 'de tekrarlar; dalga/animasyon döngüsü.

$(\cos \theta, \sin \theta)$ neden birim vektördür?

Uzunluğu hesaplayalım:

$$\|(\cos \theta, \sin \theta)\| = \sqrt{\cos^2 \theta + \sin^2 \theta} = \sqrt{1} = 1.$$

İşte bu yüzden $(\cos \theta, \sin \theta)$ her zaman *birim* bir yön verir — kamera bakış yönü üretirken (yaw/pitch) tam da bundan faydalanırız.

5.4 Tanjant ve atan2: Noktadan Açığa

Bazen tersine gitmek gerekir: elimizde bir yön vektörü var, açısını istiyoruz. Bunun için `atan2(y, x)` kullanılır — sıradan `atan`'dan farkı, dört bölgeyi (quadrant) doğru ayırt etmesi ve $-\pi$ ile $+\pi$ arasında doğru açıyı vermesidir.

```

1 // Direction from player to target -> angle to face it
2 glm::vec2 toTarget = targetPos - playerPos;
3 float facingAngle = std::atan2(toTarget.y, toTarget.x); // radians, -PI..PI
4
5 // Turn it back into a facing direction vector:
6 glm::vec2 facing(std::cos(facingAngle), std::sin(facingAngle));

```

Dikkat

Her zaman `atan2(y, x)` kullan, `atan(y/x)` değil. `atan` noktanın hangi çeyrekte olduğunu bilemez (ör. $(1,1)$ ile $(-1,-1)$ aynı oranı verir) ve $x = 0$ 'da sifra bölme yaşarsın. `atan2` bu iki sorunu da çözer.

5.5 Trigonometri Neden Döndürme Matrisinin İçinde?

Şimdi her şey birleşiyor. Bir (x, y) noktasını θ açısıyla döndürünce yeni konum şudur:

$$x' = x \cos \theta - y \sin \theta, \quad y' = x \sin \theta + y \cos \theta$$

Bu iki satır tam olarak Z-ekseni döndürme matrisinin ilk iki satırıdır (bir sonraki bölümde göreceğiz). Yani döndürme matrisindeki bütün o $\sin/\cos$ değerleri, "noktayı birim çember boyunca θ kadar kaydır" demenin matris dilindeki karşılığıdır.

```

1 // Rotating a 2D point by hand (what the rotation matrix does internally)
2 glm::vec2 rotate2D(glm::vec2 p, float theta) {
3     float s = std::sin(theta), c = std::cos(theta);
4     return glm::vec2(p.x * c - p.y * s, // x' = x cos - y sin

```

```

5         p.x * s + p.y * c); // y' = x sin + y cos
6     }

```

Bilgi

Trigonometri ile animasyon: $\sin$ ve $\cos$ $[-1,1]$ arasında yumuşakça salındığı için "nefes alma", yukarı-aşağı süzülme, yanıp sönme gibi efektlerin doğal kaynağıdır. `float offset = std::sin glfwGetTime() * hız) * genlik;` tek satırla pürüzsüz bir salınım verir. Aynı fikir, GLSL'de dalga/su efektlerinin de temelidir.

İpucu

Grafikte trigonometriyi üç yerde göreceksin: (1) **döndürme** (matrislerdeki $\sin/\cos$), (2) **yön üretme** (yaw/pitch $\rightarrow$ birim vektör, kamera bölümü), (3) **periyodik hareket** (yörünge, dalga, salınım). Üçü de bu bölümdeki tek fikre dayanır: açı $\rightarrow$ çember üzerinde nokta.

6. Vektörler

Vektör, hem büyüklüğü (uzunluk) hem yönü olan bir niceliktir. Grafikte konumları, yönleri, renkleri, hızları temsil eder. 3B'de üç bileşeni vardır: $\mathbf{v} = (x, y, z)$.

6.1 Temel İşlemler

İşlem	Formül	Anlamı
Toplama	$\mathbf{a} + \mathbf{b} = (a_x + b_x, \dots)$	Yer değiştirmelerin zinciri.
Skalerle çarpma	$s\mathbf{a} = (sa_x, \dots)$	Uzatma/kısaltma.
Uzunluk	$\ \mathbf{a}\ = \sqrt{a_x^2 + a_y^2 + a_z^2}$	Büyüklük (norm).
Normalizasyon	$\hat{\mathbf{a}} = \mathbf{a} / \ \mathbf{a}\ $	Uzunluğu 1 olan birim yön.

```

1 #include <glm/glm.hpp>
2 #include <glm/gtc/type_ptr.hpp>
3
4 glm::vec3 a(1.0f, 2.0f, 3.0f);
5 glm::vec3 b(4.0f, 5.0f, 6.0f);
6
7 glm::vec3 sum      = a + b;           // (5, 7, 9)
8 glm::vec3 scaled   = 2.0f * a;        // (2, 4, 6)
9 float      len     = glm::length(a);  // sqrt(14) ~ 3.742
10 glm::vec3 unit     = glm::normalize(a); // length becomes 1

```

Dikkat

Bir vektörü normalize etmek onu *birim vektör* yapar (uzunluğu 1). Işık ve kamera hesaplarının neredeyse tamamı yön vektörlerinin normalize olmasını bekler; unutursan aydınlatma yanlış çıkar. `glm::normalize` ile sıfır vektörü normalize etme — 0'a bölme NaN üretir.

6.2 Nokta Çarpım (Dot Product)

İki vektörün nokta çarpımı bir *skalerdir* ve aralarındaki açıyla ilgilidir:

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

Her iki vektör de birim ise, nokta çarpım doğrudan $\cos \theta$ 'dır. Bu, grafikte altın değerindedir: bir yüzeyin ışığa ne kadar dönük olduğunu (diffuse aydınlatma) tam olarak bu belirler.

a · b işareti

Yorum

> 0 Vektörler aynı yöne bakıyor (açı < 90°).

= 0 Dik (ortogonal).

< 0 Zıt yöne bakıyor (açı > 90°).

```
1 float d = glm::dot(a, b); // 1*4 + 2*5 + 3*6 = 32
2
3 // Angle between two directions (in radians):
4 glm::vec3 u = glm::normalize(a);
5 glm::vec3 v = glm::normalize(b);
6 float angle = glm::acos(glm::dot(u, v)); // 0..PI
7
8 // Is the surface facing the light? (diffuse factor)
9 float diffuse = glm::max(glm::dot(normal, lightDir), 0.0f);
```

6.3 Çapraz Çarpım (Cross Product)

İki vektörün çapraz çarpımı, **her ikisine de dik** yeni bir vektördür. Yüzey normallerini ve kamera baz eksenlerini bulmakta vazgeçilmezdir.

$$\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

```
1 glm::vec3 n = glm::cross(a, b); // vector perpendicular to both a and b
2
3 // Surface normal of a triangle (from two edges):
4 glm::vec3 edge1 = p1 - p0;
5 glm::vec3 edge2 = p2 - p0;
6 glm::vec3 normal = glm::normalize(glm::cross(edge1, edge2));
```

Bilgi

Çapraz çarpımın yönü *sağ el kuralına* uyar ve sıra önemlidir: $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$. Yüzey normalinin "dışarı" mı "içeri" mi baktığı, köşe sıralamana (winding order) bağlıdır — bu, arka yüz eleme (back-face culling) için kritiktir.

7. Matrisler ve Dönüşümler

Matris, bir sayı ızgarasıdır ve grafikte **dönüşümleri** temsil eder. Bir noktayı bir matrisle çarpmak onu öteler, döndürür veya ölçekler. 3B grafikte 4×4 matrisler kullanılır (nedenini homojen koordinatlarda göreceğiz).

7.1 Matris Çarpımı ve Birim Matris

Bir dönüşümü bir vektöre uygulamak, $\text{matris} \times \text{vektör}$ çarpımıdır. Birden fazla dönüşümü birleştirmek ise $\text{matris} \times \text{matris}$ çarpımıdır. **Birim matris $\mathbf{I}$** hiçbir şeyi değiştirmez (1 ile çarpmak gibi):

$$\mathbf{I} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Sayısal örnek: 2×2 matris $\times$ vektör

Bir noktayı 90° döndüren matris ile $(1, 0)$ noktasını çarpalım:

$$\begin{pmatrix} \cos 90^\circ & -\sin 90^\circ \\ \sin 90^\circ & \cos 90^\circ \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$(1, 0)$ noktası $(0, 1)$ 'e döndü — yani saat yönünün tersine 90° . İşte tüm dönüşümler bu $\text{sıra} \times \text{sütun}$ çarpımından ibarettir.

```
1 #include <glm/gtc/matrix_transform.hpp>
2
3 glm::mat4 identity(1.0f); // constructing with 1.0f => IDENTITY (very common gotcha!)
```

Dikkat

`glm::mat4 m;` demek matrisi *sıfırlamaz* ve birim yapmaz; inilendirilmemiş kalabilir. Her zaman `glm::mat4 m(1.0f)` yazarak açıkça birim matris kur. Sıfır matrisle çarpılan her nokta orijine çöker ve ekranda hiçbir şey görünmez.

Dikkat

Matris çarpımı **değişmeli değildir**: $\mathbf{AB} \neq \mathbf{BA}$. "Önce döndür sonra öte" ile "önce öte sonra döndür" tamamen farklı sonuç verir. Sıra her şeydir.

7.2 Öteleme (Translation)

Bir noktayı (t_x, t_y, t_z) kadar kaydırır. Homojen 4×4 formu:

$$\mathbf{T} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

```
1 glm::mat4 translation =
2     glm::translate(glm::mat4(1.0f), glm::vec3(2.0f, 1.0f, 0.0f));
3 // Moves the object +2 on X and +1 on Y.
```

7.3 Ölçekleme (Scale)

Her eksen bağımsız büyütür/küçültür:

$$\mathbf{S} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

```
1 glm::mat4 scaling = glm::scale(glm::mat4(1.0f), glm::vec3(2.0f)); // 2x on every axis
```

7.4 Döndürme (Rotation)

Bir eksen etrafında θ açısıyla döndürür. Örneğin Z eksen etrafında:

$$\mathbf{R}_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

```
1 // glm::radians converts degrees -> radians (GLM expects radians!)
2 glm::mat4 rotation = glm::rotate(glm::mat4(1.0f),
3   glm::radians(45.0f),           // angle
4   glm::vec3(0.0f, 0.0f, 1.0f)); // around Z axis
```

Dikkat

GLM'nin trigonometrik ve döndürme fonksiyonları **radyan** bekler, derece değil. `glm::rotate(m, 90.0f, axis)` yazarsan 90 *radyan* dönersin (yani 5157 derece). Daima `glm::radians(90.0f)` kullan.

7.5 Dönüşümleri Birleştirme

Birden fazla dönüşümü tek matriste toplayabiliriz. Kritik nokta: GLM'de (ve OpenGL'de) çarpım **sağdan sola** uygulanır. Yani kodda önce yazdığın matris, noktaya en *son* uygulanır.

```
1 // Goal: first scale the object, then rotate it, and finally translate it.
2 // Application order to a vertex v: T * R * S * v (S applies to v first)
3 glm::mat4 model(1.0f);
4 model = glm::translate(model, position);           // applied 3rd
5 model = glm::rotate(model, glm::radians(angle), axis); // applied 2nd
6 model = glm::scale(model, scale);                 // applied 1st (closest to v)
7 // Result: model = T * R * S
```

Sağdan sola sezgisi

`model = T; model *= R; model *= S;` zinciri $\mathbf{T} \cdot \mathbf{R} \cdot \mathbf{S}$ üretir. Bir köşe $\mathbf{v}$ bununla çarpılınca önce $\mathbf{S}$ (ölçek), sonra $\mathbf{R}$ (döndür), en son $\mathbf{T}$ (öte) etkir. Doğru sıra: önce yerel şekli düzenle (ölçek/döndür), en son dünyaya yerleştir (öte). Tersini yaparsan nesne orijin etrafında yörüngeye girer.

8. Homojen Koordinatlar

Neden 3×3 değil de 4×4 matris? Çünkü **öteleme** bir matris çarpımıyla ifade edilemez — düz matris çarpımı orijini sabit tutar. Çözüm: noktalara dördüncü bir bileşen w ekleyip (x, y, z, w) yapmak. Buna *homojen koordinatlar* denir.

w değeri

Anlamı

$w = 1$ Bir **nokta** (konum). Öteleme onu etkiler.

$w = 0$ Bir **yön** (vektör). Öteleme onu etkilemez, sadece döndürme/ölçek.

```
1 glm::vec4 point      = glm::vec4(pos, 1.0f); // w=1: a position, translation affects it
2 glm::vec4 direction = glm::vec4(dir, 0.0f); // w=0: a direction, translation ignored
3
4 glm::mat4 model = /* ... */;
5 glm::vec3 worldPos = glm::vec3(model * point); // translated + rotated + scaled
6 glm::vec3 worldDir = glm::vec3(model * direction); // only rotated + scaled
```

Bilgi

Bu ayrım çok işe yarar: bir konum noktasını dünyaya taşıırken ($w=1$) öteleme uygulanmalı, ama bir yön vektörünü (ör. ışık yönü, yüzey normali, $w=0$) taşıırken öteleme *uygulanmamalı* — yönün başladığı yer önemsizdir.

8.1 Perspektif Bölme

Perspektif projeksiyondan sonra w artık 1 olmayabilir. GPU, clip uzayından NDC'ye geçerken her bileşeni w 'ye böler:

$$(x, y, z, w) \rightarrow \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w}\right)$$

İşte "uzaktakiler küçük görünür" etkisi buradan gelir: uzak nesnelerde w büyüktür, bölünce koordinatlar küçülür. Bu bölmeyi GPU otomatik yapar; biz sadece doğru projeksiyon matrisini vermekle yükümlüüz.

9. View ve Projection: Kamera Matematiği

9.1 View Matrisi (LookAt)

Aslında "kamerayı hareket ettirmek" diye bir şey yoktur; bunun yerine *tüm dünyayı* kameranın tersine hareket ettiririz. View matrisi bunu yapar. GLM'nin lookAt fonksiyonu üç şeyle bir kamera kurar:

Parametre

Anlamı

eye (position) Kameranın dünyadaki konumu.

center (target) Kameranın baktığı nokta.

up "Yukarı" yönü (genelde $(0, 1, 0)$).


```

1 glm::vec3 cameraPos    = glm::vec3(0.0f, 0.0f, 3.0f);
2 glm::vec3 cameraTarget = glm::vec3(0.0f, 0.0f, 0.0f);
3 glm::vec3 worldUp      = glm::vec3(0.0f, 1.0f, 0.0f);
4
5 glm::mat4 view = glm::lookAt(cameraPos, cameraTarget, worldUp);

```

Perde arkası

lookAt içeride kamera için üç dik birim eksen kurar: *forward* (target-eye), *right* (up × forward) ve *true up* (forward × right). Bu çapraz çarpımlar tam da 6. bölümde öğrendiğimiz işlemdir. Sonra bu eksenlerle dünyayı kamera uzayına döndürüp öteler.

9.2 Projection Matrisi

Projeksiyon matrisi 3B sahneyi 2B ekrana nasıl "yansıtacağımızı" belirler ve görünür hacmi (frustum) tanımlar. İki tür vardır:

Perspektif

Ortografik

Uzaktakiler küçülür (gerçekçi, 3B oyunlar). Boyut mesafeden bağımsız (CAD, 2B, izometrik).

Görünür hacim bir *piramit kesiği* (frustum). Görünür hacim bir *dikdörtgen prizma*.

```

1 // Perspective: FOV, aspect ratio, near and far planes
2 float aspect = (float)width / (float)height;
3 glm::mat4 projection = glm::perspective(glm::radians(45.0f), // vertical field of view
4                                         aspect,                // width / height
5                                         0.1f,                  // near (must be > 0!)
6                                         100.0f);               // far
7
8 // Orthographic: left, right, bottom, top, near, far
9 glm::mat4 ortho = glm::ortho(0.0f, 800.0f, 0.0f, 600.0f, -1.0f, 1.0f);

```

Dikkat

Perspektifte near düzlemi kesinlikle **0'dan büyük** olmalıdır (ör. 0.1). 0 verersen derinlik tamponu (z-buffer) hassasiyeti bozulur ve "z-fighting" (titreşen yüzeyler) yaşarsın. Ayrıca near/far aralığını gereksiz geniş tutma; hassasiyet near'a yakın toplanır.

9.3 Hepsini Birleştirmek

Artık MVP zincirini tamamlayabiliriz. Vertex shader'da her köşeyi şöyle dönüştüreceğiz:

```

1 // On the C++ side we build three matrices and send them as uniforms:
2 glm::mat4 model    = /* object's placement in the world */;
3 glm::mat4 view     = glm::lookAt(cameraPos, cameraTarget, worldUp);
4 glm::mat4 projection = glm::perspective(glm::radians(45.0f), aspect, 0.1f, 100.0f);
5
6 // Inside the GLSL vertex shader:
7 //   gl_Position = projection * view * model * vec4(aPos, 1.0);

```

İpucu

Kısım 2 bitti. Artık bir noktanın modelden ekrana yolculuğunun her adımının matematiğini biliyorsun. Bundan sonrası bu matematiği çalışan bir pencerede canlandırmak: GLFW ile pencere açıp OpenGL ile çizmek.

KISIM 3

GLFW: Pencere, Bağlam ve Girdi

GLFW (Graphics Library Framework), çok platformlu, hafif bir C kütüphanesidir. Tek işi vardır ama onu çok iyi yapar: bir OpenGL **bağlamı (context)** olan pencere açmak, olayları (klavye, fare, boyutlandırma) yönetmek ve tamponları değiştirmek. GLFW hiçbir şey *çizmez* — çizim OpenGL'in işidir. GLFW ise üzerine çiziceğin tuvali ve girdi sistemini sağlar.

10. Kurulum ve Derleme

Bir OpenGL projesi tipik olarak üç parçadan oluşur:

Bileşen	Görevi
GLFW	Pencere + bağlam + girdi.
GLAD (veya GLEW)	OpenGL fonksiyon işaretçilerini çalışma anında yükler.
GLM	Vektör/matris matematiği (başlık-only).

Bilgi

GLAD neden gerekli? OpenGL'in modern fonksiyonları (ör. `glCreateShader`) sürücüde bulunur ve adresleri çalışma anında öğrenilmelidir. GLAD bu adresleri senin için yükler. GLAD kodunu <https://glad.dav1d.de> sitesinden üretirsin: API=OpenGL, Version=3.3, Profile=Core seçip indir.

10.1 Linux'ta Paketlerin Kurulumu

```

1 # Debian / Ubuntu
2 $ sudo apt install libglfw3-dev libglm-dev cmake g++
3 # Download GLAD from glad.dav1d.de and add it (glad.c + include/) to the project
4
5 # Fedora
6 $ sudo dnf install glfw-devel glm-devel cmake gcc-c++
7
8 # Arch
9 $ sudo pacman -S glfw glm cmake

```

10.2 CMake ile Proje Yapısı

Önerilen dizin düzeni:

```

1 project/
2 |-- CMakeLists.txt
3 |-- src/
4 |   |-- main.cpp
5 |-- glad/
6 |   |-- src/glad.c
7 |   |-- include/glad/glad.h
8 |   |-- include/KHR/khrplatform.h
9 |-- shaders/
10 |   |-- basic.vert
11 |   |-- basic.frag

```

```

1 # CMakeLists.txt
2 cmake_minimum_required(VERSION 3.16)
3 project(GLFWGuide LANGUAGES C CXX)
4
5 set(CMAKE_CXX_STANDARD 17)
6 set(CMAKE_CXX_STANDARD_REQUIRED ON)
7
8 find_package(GLFW3 REQUIRED)
9 find_package(OpenGL REQUIRED)
10
11 add_executable(app
12     src/main.cpp
13     glad/src/glad.c)      # include GLAD in the build
14
15 target_include_directories(app PRIVATE glad/include)
16 target_link_libraries(app PRIVATE glfw OpenGL::GL ${CMAKE_DL_LIBS})
17 # ${CMAKE_DL_LIBS}: GLAD may need libdl for dlopen

```

```

1 # Build
2 $ cmake -S . -B build
3 $ cmake --build build
4 $ ./build/app

```

Elle derleme

CMake istemiyorsan tek satırla da derleyebilirsin: `g++ -std=c++17 src/main.cpp glad/src/glad.c -Iglad/include -lglfw -lGL -ldl -o app`

11. İlk Pencere

Şimdi en küçük tam GLFW programını yazalım. Bu program bir pencere açar, koyu turkuaz bir arka planla temizler ve kullanıcı kapatana kadar açık tutar. Her satırı yorumluyoruz.

```

1 #include <glad/glad.h>    // GLAD must be included BEFORE GLFW!
2 #include <GLFW/glfw3.h>
3 #include <iostream>
4
5 // GLFW reports internal errors through this callback
6 void errorCallback(int code, const char* description) {
7     std::cerr << "GLFW error " << code << ": " << description << '\n';
8 }

```

```

9
10 int main() {
11     // 1) Register the error callback (before glfwInit so init errors are caught)
12     glfwSetErrorCallback(errorCallback);
13
14     // 2) Initialize GLFW
15     if (!glfwInit()) {
16         std::cerr << "Failed to initialize GLFW\n";
17         return -1;
18     }
19
20     // 3) Request OpenGL 3.3 Core profile
21     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
22     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
23     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
24 #ifdef __APPLE__
25     glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE); // required on macOS
26 #endif
27
28     // 4) Create the window (width, height, title, monitor, share)
29     GLFWwindow* window = glfwCreateWindow(800, 600, "First Window", nullptr, nullptr);
30     if (!window) {
31         std::cerr << "Failed to create window\n";
32         glfwTerminate();
33         return -1;
34     }
35
36     // 5) Make this window's OpenGL context current on the calling thread
37     glfwMakeContextCurrent(window);
38
39     // 6) Load OpenGL function pointers via GLAD (AFTER the context is current!)
40     if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress)) {
41         std::cerr << "Failed to initialize GLAD\n";
42         return -1;
43     }
44
45     // 7) Set the region OpenGL renders into
46     glViewport(0, 0, 800, 600);
47
48     // 8) MAIN LOOP: run until the window is asked to close
49     while (!glfwWindowShouldClose(window)) {
50         // Clear the background
51         glClearColor(0.2f, 0.3f, 0.3f, 1.0f); // dark teal
52         glClear(GL_COLOR_BUFFER_BIT);
53
54         // Swap the front/back buffers (present the frame)
55         glfwSwapBuffers(window);
56         // Process pending events (keyboard, mouse, close)
57         glfwPollEvents();
58     }
59
60     // 9) Release resources
61     glfwDestroyWindow(window);
62     glfwTerminate();
63     return 0;
64 }

```

Dikkat

glad.h her zaman glfw3.h'den **önce** dahil edilmelidir, çünkü GLAD OpenGL başlıklarını kendi tanımlar ve GLFW bunu algılar. Sıra tersse "gl.h already included" hataları alırsın. Ayrıca GLAD'i *ancak* glfwMakeContextCurrent'tan sonra yükle — fonksiyonlar bir bağlam olmadan çözülemez.

11.1 Ana Döngünün Anatomisi

Her gerçek zamanlı uygulamanın kalbi bu döngüdür. Her tur bir *karedir*:

Adım	Çağrı
Kapatma kontrolü	glfwWindowShouldClose(window)
Girdi işle	processInput() (bizim fonksiyon)
Ekranı temizle	glClear(...)
Çizim	glDrawArrays / glDrawElements
Tamponları değiştir	glfwSwapBuffers(window)
Olayları işle	glfwPollEvents()

12. Callback'ler ve Olay İşleme

GLFW olayları (pencere boyutu değişti, tuşa basıldı, fare oynadı) **callback** fonksiyonlarıyla bildirir. Sen bir fonksiyon yazar, onu GLFW'ye kaydedersin; GLFW uygun olay olduğunda onu çağırır.

12.1 Pencere Boyutlandırma

Kullanıcı pencereyi yeniden boyutlandırınca viewport'u güncellemezsen çizimin gerilir. Bunu bir framebuffer-size callback ile çözeriz:

```

1 // GLFW calls this whenever the framebuffer (drawable area) is resized
2 void framebufferSizeCallback(GLFWwindow* window, int width, int height) {
3     glViewport(0, 0, width, height); // update OpenGL's drawing region
4 }
5
6 int main() {
7     // ... window setup ...
8     glfwSetFramebufferSizeCallback(window, framebufferSizeCallback);
9     // ...
10 }
```

Neden framebuffer boyutu?

Yüksek-DPI (Retina) ekranlarda pencere boyutu (ekran birimi) ile framebuffer boyutu (gerçek piksel) farklı olabilir. glViewport gerçek pikselleri ister, o yüzden *framebuffer* boyutu callback'ini kullanırsınız, pencere boyutunu değil.

12.2 Klavye Girdisi: İki Yöntem

Klavyeyi iki şekilde okuyabilirsin. Sürekli hareket için *polling*, tek seferlik olaylar için *callback* uygundur.

```

1 // METHOD 1: Polling -- query state every frame (ideal for continuous movement)
2 void processInput(GLFWwindow* window) {
3     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
4         glfwSetWindowShouldClose(window, true); // quit on ESC
5
6     if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
7         std::cout << "Moving forward...\n";
8 }
9
10 // METHOD 2: Callback -- event based (ideal for single presses, menus, shortcuts)
11 void keyCallback(GLFWwindow* win, int key, int scancode, int action, int mods) {
12     if (key == GLFW_KEY_SPACE && action == GLFW_PRESS)
13         std::cout << "Fire!\n"; // once, at the moment of press
14     if (key == GLFW_KEY_F && action == GLFW_PRESS && (mods & GLFW_MOD_CONTROL))
15         std::cout << "Ctrl+F pressed\n"; // modifier keys
16 }
17
18 int main() {
19     // ...
20     glfwSetKeyCallback(window, keyCallback);
21     while (!glfwWindowShouldClose(window)) {
22         processInput(window); // polling happens here
23         // ... draw ...
24     }
25 }

```

action değeri

Anlamı

GLFW_PRESS	Tuşa yeni basıldı.
GLFW_RELEASE	Tuş bırakıldı.
GLFW_REPEAT	Tuş basılı tutuluyor (tekrar).

İpucu

Karakter/oyuncu hareketi için **polling** kullan (`glfwGetKey`): her karede "W basılı mı?" diye sor, basılıysa hareket ettir. Menü seçimi, zıplama, ateş gibi *tek seferlik* eylemler için **callback** kullan — böylece tuş basılı tutulunca eylem tekrarlanmaz.

12.3 Fare Girdisi

Fare konumu, tuşları ve tekerleği için ayrı callback'ler vardır. FPS kamera için fare hareketini kullanacağız (Kısım 4).

```

1 // Mouse movement (cursor position, in screen pixels)
2 void cursorPosCallback(GLFWwindow* window, double xpos, double ypos) {
3     std::cout << "Cursor: " << xpos << ", " << ypos << "\n";
4 }
5
6 // Scroll wheel (horizontal and vertical offsets)
7 void scrollCallback(GLFWwindow* window, double xoffset, double yoffset) {
8     std::cout << "Scroll: " << yoffset << "\n"; // ideal for zoom
9 }
10

```

```

11 // Mouse buttons
12 void mouseButtonCallback(GLFWwindow* win, int button, int action, int mods) {
13     if (button == GLFW_MOUSE_BUTTON_LEFT && action == GLFW_PRESS)
14         std::cout << "Left click\n";
15 }
16
17 int main() {
18     // ...
19     glfwSetCursorPosCallback(window, cursorPosCallback);
20     glfwSetScrollCallback(window, scrollCallback);
21     glfwSetMouseButtonCallback(window, mouseButtonCallback);
22
23     // For an FPS camera: hide the cursor and lock it to the window
24     glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);
25 }

```

13. Zaman, Delta Time ve FPS

Farklı bilgisayarlar farklı hızlarda kare üretir. Hareketi kare sayısına bağlarsan, hızlı makinede nesne uçar, yavaşta sürünür. Çözüm: her hareketi **delta time** (iki kare arası geçen süre) ile çarpmak. Böylece hız donanımdan bağımsız olur.

```

1 float deltaTime = 0.0f; // duration of the last frame (seconds)
2 float lastFrame = 0.0f; // timestamp of the previous frame
3
4 int main() {
5     // ...
6     while (!glfwWindowShouldClose(window)) {
7         float currentFrame = (float)glfwGetTime(); // seconds since program start
8         deltaTime = currentFrame - lastFrame;
9         lastFrame = currentFrame;
10
11         // Scale movement by delta time (units: distance per second)
12         float velocity = 2.5f * deltaTime;
13         cameraPos += velocity * direction; // now independent of FPS!
14
15         // ... draw ...
16         glfwSwapBuffers(window);
17         glfwPollEvents();
18     }
19 }

```

13.1 Basit FPS Sayacı

```

1 double previousSeconds = glfwGetTime();
2 int frameCount = 0;
3
4 // Inside the main loop:
5 double currentSeconds = glfwGetTime();
6 frameCount++;
7 if (currentSeconds - previousSeconds >= 1.0) { // one second elapsed
8     std::cout << "FPS: " << frameCount << '\n';
9     frameCount = 0;
10    previousSeconds = currentSeconds;
11 }

```


İpucu

`glfwSwapInterval(1)` ile VSync açarak FPS'i ekran yenileme hızına sabitleyebilir, GPU'yu boşuna yormazsın. Performans ölçerken `glfwSwapInterval(0)` ile kapat ki gerçek kare süresini görebilesin.

KISIM 4

OpenGL ile Çizim

Artık pencere ve girdi hazır. Bu kısımda gerçek çizime geçiyoruz: köşe verisini GPU'ya yükleyip, shader'larla işleyip, ekrana üçgen, kare, dokulu yüzey ve nihayet aydınlatılmış 3B küp çizeceğiz. Kısım 2'nin matematiği burada canlanacak.

14. Shader'lar ve GLSL Dili Derinlemesine

Shader, GPU üzerinde çalışan küçük bir programdır. Modern OpenGL'de shader olmadan tek bir piksel bile çizemezsin — bu yüzden shader'ları gerçekten anlamak zorunludur. Shader'lar **GLSL** (OpenGL Shading Language) ile yazılır; C'ye çok benzeyen ama grafik için özelleştirilmiş bir dildir.

14.1 Shader Türleri ve Boru Hattındaki Yeri

Shader	Zorunlu?	Ne yapar?
Vertex	Evet	Her <i>köşe</i> için bir kez çalışır; konumu clip uzayına koyar, öznitelikleri sonraki aşamaya geçirir.
Fragment	Evet	Her <i>fragment</i> (aday piksel) için bir kez çalışır; nihai rengi üretir.
Geometry	Hayır	Primitive'leri işler; yeni geometri üretebilir (ör. tek noktadan quad).
Tessellation	Hayır	Yüzeyleri dinamik olarak alt bölümlere ayırır (arazi, LOD).
Compute	Hayır	Grafikten bağımsız genel amaçlı GPU hesabı (parçacık, fizik).

Bilgi

Bu rehberde yalnızca zorunlu ikiliyi — **vertex** ve **fragment** — kullanıyoruz. İkisi bir *shader programı* altında derlenip bağlanır. Vertex'in out değişkenleri, fragment'in in değişkenlerine *isim* üzerinden bağlanır ve arada rasterizasyon interpolasyon yapar.

14.2 Bir Shader'ın İskeleti

```

1 #version 330 core    // GLSL version: OpenGL 3.3 Core -> GLSL 330
2
3 // (input/output/uniform declarations go here)
4
5 void main() {
6     // shader body -- runs once per vertex/fragment
7 }
```

Sürüm eşleşmesi

#version 330 core, OpenGL 3.3 Core bağlamıyla eşleşir. GLSL sürüm numarası OpenGL'inkinden farklıdır: GL 3.3 → GLSL 330, GL 4.6 → GLSL 460. #version satırı dosyanın ilk satırı olmalıdır — önünde yorum bile olamaz.

14.3 Veri Tipleri

Tip	Açıklama
<code>float</code> , <code>int</code> , <code>bool</code>	Skaler tipler.
<code>vec2</code> , <code>vec3</code> , <code>vec4</code>	2/3/4 bileşenli float vektör (konum, renk, UV).
<code>ivec3</code> , <code>bvec3</code>	int / bool vektörleri.
<code>mat2</code> , <code>mat3</code> , <code>mat4</code>	Kare matrisler (dönüşümler).
<code>sampler2D</code>	Bir dokuya erişim tutamacı.

```

1 vec3 color    = vec3(1.0, 0.5, 0.2);      // orange
2 vec4 position = vec4(color, 1.0);         // vec3 + w -> vec4 (constructor stitching)
3 vec3 uniform3 = vec3(0.5);               // (0.5, 0.5, 0.5) -- all the same
4 mat4 identity = mat4(1.0);               // identity matrix

```

14.4 Swizzling: Vektör Bileşenlerine Erişim

GLSL'in en sevilen özelliği **swizzling**'dir: vektör bileşenlerini harflerle seçip yeniden düzenleyebilirsiniz. `x,y,z,w` (konum), `r,g,b,a` (renk) ve `s,t,p,q` (doku) aynı bileşenlere farklı isimlerdir.

```

1 vec4 v = vec4(1.0, 2.0, 3.0, 4.0);
2
3 float x    = v.x;          // 1.0
4 vec2 xy    = v.xy;         // (1.0, 2.0)
5 vec3 bgr   = v.bgr;        // (3.0, 2.0, 1.0) -- reverse color channels
6 vec3 all   = v.xxx;        // (1.0, 1.0, 1.0) -- repetition is allowed
7 v.xy = v.yx;              // swap components

```

İpucu

Swizzling sadece okuma değil, atama için de çalışır. `fragColor.rgb *= 0.5;` tüm renk kanallarını yarıya indirirken alfa'yı korur.

14.5 Giriş, Çıkış ve Nitelikler (in / out / uniform)

Nitelik	Anlamı
in	Bu shader'a <i>gelen</i> veri. Vertex'te: VBO'dan öznitelik. Fragment'te: vertex'ten interpolasyonla gelen değer.
out	Bu shader'dan <i>çıkan</i> veri. Vertex'te: sonraki aşamaya. Fragment'te: framebuffer'a (renk).
uniform	CPU'dan (C++) gelen, tüm çizim boyunca <i>sabit</i> global değer.
layout(location=N)	Bir in özniteliğine sabit numara atar; glVertexAttribPointer'daki index ile eşleşir.

```

1 // VERTEX SHADER -- example showing all three qualifiers
2 #version 330 core
3 layout (location = 0) in vec3 aPos;    // position attribute from the VBO
4 layout (location = 1) in vec3 aColor;  // color attribute from the VBO
5
6 out vec3 vColor;                      // passed to the fragment shader (gets interpolated)
7 uniform mat4 mvp;                     // constant transform matrix from the CPU
8
9 void main() {
10     gl_Position = mvp * vec4(aPos, 1.0); // required output: clip-space position
11     vColor = aColor;                    // forward color to the next stage
12 }

```

```

1 // FRAGMENT SHADER -- receives the vertex shader's out as in
2 #version 330 core
3 in vec3 vColor;                       // interpolated color from the vertex shader
4 out vec4 FragColor;                   // final color written to the framebuffer
5 uniform float alpha;                  // transparency from the CPU
6
7 void main() {
8     FragColor = vec4(vColor, alpha);
9 }

```

Dikkat

Vertex'teki out ile fragment'teki in **aynı ada ve tipe** sahip olmalıdır — eşleşme isim üzerindendir. Adı yanlış yazarsan fragment o değeri hiç almaz ve genelde siyah ekranla karşılaşırısın.

14.6 Yerleşik Değişkenler ve Fonksiyonlar

Değişken	Shader	Rolü
gl_Position	Vertex	Zorunlu çıktı: köşenin clip uzayı konumu (vec4).
gl_FragCoord	Fragment	Fragment'in ekran piksel koordinatı (salt-okunur).
gl_VertexID	Vertex	O anki köşenin indeksi.

```

1 // A helper function + built-ins in a fragment shader
2 float luminance(vec3 c) {

```

```

3   return dot(c, vec3(0.2126, 0.7152, 0.0722)); // grayscale weights
4 }
5
6 void main() {
7     vec3 texColor = texture(image, uv).rgb;
8     float gray    = luminance(texColor);
9     vec3 result   = mix(texColor, vec3(gray), 0.5); // 50% desaturated
10    FragColor = vec4(result, 1.0);
11 }

```

Bilgi

Fragment shader'da `discard;` anahtar kelimesi o fragment'i tamamen atar (hiç yazılmaz) — ağaç yaprağı gibi delikli dokularda saydam pikselleri elemek için kullanılır. GLSL'de `if/for` vardır ama GPU'da dallanma pahalı olabilir; mümkünse `mix/step` gibi dalsız işlevleri tercih et.

İpucu

Shader'ları anlamamanın en hızlı yolu *oynamaktır*: bir zaman uniform'u alıp `sin` ile renkleri dalgalandır. <https://www.shadertoy.com> tam da bu deneyler için bir oyun alanıdır.

15. VBO, VAO ve İlk Üçgen

Modern OpenGL'de üçgen çizmek için üç kavramı bilmemiz gerekir:

Nesne

Görevi

VBO (Vertex Buffer Object)	Köşe verisini (konum, renk...) GPU belleğine koyar.
VAO (Vertex Array Object)	Bu verinin <i>nasıl yorumlanacağını</i> kaydeder (öznitelik düzeni).
Shader Program	Vertex + fragment shader'ın derlenmiş, bağlanmış hali.

15.1 Shader'ları Yazmak

```

1 // basic.vert -- VERTEX SHADER
2 #version 330 core
3 layout (location = 0) in vec3 aPos;    // attribute 0: position
4
5 void main() {
6     // Put the position straight into clip space (w=1). No matrix -- vertices are in NDC.
7     gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0);
8 }

```

```

1 // basic.frag -- FRAGMENT SHADER
2 #version 330 core
3 out vec4 FragColor;    // this fragment's final color
4
5 void main() {
6     FragColor = vec4(1.0, 0.5, 0.2, 1.0); // orange (RGBA)
7 }

```

15.2 Shader Derleme ve Bağlama

Shader kaynak kodu bir string'dir; onu GPU'da derleyip programa bağlamalıyız. Hata kontrolü şart — yoksa siyah ekranla baş başa kalırsın.

```

1 unsigned int compileShader(unsigned int type, const char* source) {
2     unsigned int shader = glCreateShader(type); // GL_VERTEX_SHADER / GL_FRAGMENT_SHADER
3     glShaderSource(shader, 1, &source, nullptr);
4     glCompileShader(shader);
5
6     int success;
7     glGetShaderiv(shader, GL_COMPILE_STATUS, &success);
8     if (!success) {
9         char log[512];
10        glGetShaderInfoLog(shader, 512, nullptr, log);
11        std::cerr << "Shader compile error:\n" << log << "\n";
12    }
13    return shader;
14 }
15
16 unsigned int createShaderProgram(const char* vertexSrc, const char* fragmentSrc) {
17     unsigned int vs = compileShader(GL_VERTEX_SHADER, vertexSrc);
18     unsigned int fs = compileShader(GL_FRAGMENT_SHADER, fragmentSrc);
19
20     unsigned int program = glCreateProgram();
21     glAttachShader(program, vs);
22     glAttachShader(program, fs);
23     glLinkProgram(program);
24
25     int success;
26     glGetProgramiv(program, GL_LINK_STATUS, &success);
27     if (!success) {
28         char log[512];
29         glGetProgramInfoLog(program, 512, nullptr, log);
30         std::cerr << "Program link error:\n" << log << "\n";
31     }
32     // The compiled shaders are now embedded in the program; delete the singles
33     glDeleteShader(vs);
34     glDeleteShader(fs);
35     return program;
36 }

```

15.3 Köşe Verisi, VAO/VBO Kurulumu ve Çizim

```

1 // Three vertices in NDC: bottom-left, bottom-right, top-center
2 float vertices[] = {
3     -0.5f, -0.5f, 0.0f, // bottom left
4     0.5f, -0.5f, 0.0f, // bottom right
5     0.0f, 0.5f, 0.0f // top center
6 };
7
8 unsigned int VAO, VBO;
9 glGenVertexArrays(1, &VAO);
10 glGenBuffers(1, &VBO);
11
12 glBindVertexArray(VAO); // 1) bind the VAO (recording begins)
13
14 glBindBuffer(GL_ARRAY_BUFFER, VBO); // 2) bind the VBO

```

```

15 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW); // 3) upload
16
17 // 4) Describe attribute 0: 3 floats, position; stride = 3 floats; offset = 0
18 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
19 glEnableVertexAttribArray(0); // 5) enable attribute 0
20
21 // (optional) unbind -- the VAO already recorded the layout
22 glBindBuffer(GL_ARRAY_BUFFER, 0);
23 glBindVertexArray(0);

```

```

1 // Inside the main loop -- drawing:
2 glUseProgram(shaderProgram); // use our shader
3 glBindVertexArray(VAO); // restore the layout
4 glDrawArrays(GL_TRIANGLES, 0, 3); // start at 0, draw 3 vertices

```

glVertexAttribPointer'in parametreleri

Bu fonksiyon OpenGL'de en çok kafa karıştırandır. (index, size, type, normalized, stride, offset): *index* = shader'daki location; *size* = bileşen sayısı (vec3 için 3); *stride* = bir köşenin toplam bayt boyu; *offset* = bu özniteliğin köşe içindeki başlangıç baytı.

Dikkat

Hiçbir şey görünmüyorsa kontrol listesi: (1) Shader'lar derlendi/bağlandı mı (log'a bak)? (2) glUseProgram çağrıldı mı? (3) VAO bağlı mı? (4) Köşeler NDC $[-1,1]$ içinde mi? (5) Üçgen sarım yönü yüzünden culling'e mi takıldı? (6) glClearColor rengi ile çizim rengi aynı mı (görünmez üçgen)?

15.4 Tam, Kendi Kendine Yeten İlk Üçgen Programı

Yukarıdaki parçaları tek, derlenip çalıştırılan bir dosyada birleştirelim:

```

1 #include <glad/glad.h>
2 #include <GLFW/glfw3.h>
3 #include <iostream>
4
5 const char* vertexSrc = R"(
6 #version 330 core
7 layout (location = 0) in vec3 aPos;
8 void main() { gl_Position = vec4(aPos, 1.0); }
9 )";
10
11 const char* fragmentSrc = R"(
12 #version 330 core
13 out vec4 FragColor;
14 void main() { FragColor = vec4(1.0, 0.5, 0.2, 1.0); }
15 )";
16
17 int main() {
18     glfwInit();
19     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
20     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
21     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
22
23     GLFWwindow* window = glfwCreateWindow(800, 600, "Triangle", nullptr, nullptr);
24     glfwMakeContextCurrent(window);
25     gladLoadGLLoader((GLADloadproc)glfwGetProcAddress);

```

```

26
27 // Compile shaders (inline for brevity)
28 unsigned int vs = glCreateShader(GL_VERTEX_SHADER);
29 glShaderSource(vs, 1, &vertexSrc, nullptr); glCompileShader(vs);
30 unsigned int fs = glCreateShader(GL_FRAGMENT_SHADER);
31 glShaderSource(fs, 1, &fragmentSrc, nullptr); glCompileShader(fs);
32 unsigned int program = glCreateProgram();
33 glAttachShader(program, vs); glAttachShader(program, fs);
34 glLinkProgram(program);
35 glDeleteShader(vs); glDeleteShader(fs);
36
37 float vertices[] = {
38     -0.5f, -0.5f, 0.0f,
39     0.5f, -0.5f, 0.0f,
40     0.0f, 0.5f, 0.0f
41 };
42 unsigned int VAO, VBO;
43 glGenVertexArrays(1, &VAO);
44 glGenBuffers(1, &VBO);
45 glBindVertexArray(VAO);
46 glBindBuffer(GL_ARRAY_BUFFER, VBO);
47 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
48 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
49 glEnableVertexAttribArray(0);
50
51 while (!glfwWindowShouldClose(window)) {
52     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
53         glfwSetWindowShouldClose(window, true);
54
55     glClearColor(0.2f, 0.3f, 0.3f, 1.0f);
56     glClear(GL_COLOR_BUFFER_BIT);
57
58     glUseProgram(program);
59     glBindVertexArray(VAO);
60     glDrawArrays(GL_TRIANGLES, 0, 3);
61
62     glfwSwapBuffers(window);
63     glfwPollEvents();
64 }
65
66 glDeleteVertexArrays(1, &VAO);
67 glDeleteBuffers(1, &VBO);
68 glDeleteProgram(program);
69 glfwTerminate();
70 return 0;
71 }

```

İpucu

Bu tam programı derleyip çalıştırdığında koyu turkuaz zeminde turuncu bir üçgen görürsün. OpenGL yolculuğunun "merhaba dünya"sıdır — buradan sonrası detay eklemekten ibaret.

16. Öznitelikler ve Renk İnterpolasyonu

Şimdi her köşeye *renk* de ekleyelim. Rasterizasyon, köşe renklerini üçgen yüzeyi boyunca otomatik

interpolasyon yapar — ortadaki piksel üç rengin karışımı olur.

```

1 // Each vertex: 3 position + 3 color = 6 floats
2 float vertices[] = {
3     // position      // color
4     -0.5f, -0.5f, 0.0f,  1.0f, 0.0f, 0.0f, // bottom left -- red
5     0.5f, -0.5f, 0.0f,  0.0f, 1.0f, 0.0f, // bottom right -- green
6     0.0f, 0.5f, 0.0f,  0.0f, 0.0f, 1.0f  // top           -- blue
7 };
8
9 glBindVertexArray(VAO);
10 glBindBuffer(GL_ARRAY_BUFFER, VBO);
11 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
12
13 // Attribute 0: position (offset 0)
14 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
15 glEnableVertexAttribArray(0);
16 // Attribute 1: color (offset 3 floats = 12 bytes)
17 glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float),
18                         (void*)(3 * sizeof(float)));
19 glEnableVertexAttribArray(1);

```

```

1 // VERTEX SHADER -- forward the color to the fragment shader
2 #version 330 core
3 layout (location = 0) in vec3 aPos;
4 layout (location = 1) in vec3 aColor;
5
6 out vec3 vColor; // bridge to the fragment shader (gets interpolated)
7
8 void main() {
9     gl_Position = vec4(aPos, 1.0);
10    vColor = aColor;
11 }

```

```

1 // FRAGMENT SHADER -- use the interpolated color
2 #version 330 core
3 in vec3 vColor; // comes from the vertex shader, SPREAD across the surface
4 out vec4 FragColor;
5
6 void main() {
7     FragColor = vec4(vColor, 1.0);
8 }

```

Bilgi

Vertex shader'daki out vec3 vColor, fragment shader'daki in vec3 vColor ile *isim üzerinden* eşleşir. GPU köşe değerlerini üçgen yüzeyinde perspektif-doğru interpolasyonla yayar. Sonuç: köşeleri kırmızı/yeşil/mavi olan, ortası yumuşak geçişli bir üçgen.

17. EBO ile Dikdörtgen ve Uniform'lar

17.1 EBO (Element Buffer Object)

Bir dikdörtgen iki üçgenden oluşur (6 köşe), ama sadece 4 benzersiz köşe vardır. Köşeleri tekrar etmemek için **EBO** kullanırız: köşeleri bir kez tanımlar, indekslerle hangi köşenin hangi üçgende olduğunu belirtiriz.

```

1 float vertices[] = {
2     0.5f, 0.5f, 0.0f, // 0: top right
3     0.5f, -0.5f, 0.0f, // 1: bottom right
4     -0.5f, -0.5f, 0.0f, // 2: bottom left
5     -0.5f, 0.5f, 0.0f // 3: top left
6 };
7 unsigned int indices[] = {
8     0, 1, 3, // first triangle
9     1, 2, 3 // second triangle
10 };
11
12 unsigned int VAO, VBO, EBO;
13 glGenVertexArrays(1, &VAO);
14 glGenBuffers(1, &VBO);
15 glGenBuffers(1, &EBO);
16
17 glBindVertexArray(VAO);
18 glBindBuffer(GL_ARRAY_BUFFER, VBO);
19 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
20
21 glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO); // the EBO is recorded into the VAO too
22 glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);
23
24 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
25 glEnableVertexAttribArray(0);
26
27 // Drawing: glDrawElements instead of glDrawArrays
28 glBindVertexArray(VAO);
29 glDrawElements(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0); // 6 indices

```

Dikkat

EBO bağımlı (GL_ELEMENT_ARRAY_BUFFER) VAO bağımlıdır! VAO, o an bağlı EBO'yu hatırlar. Eğer VAO'yu çözmeden önce EBO'yu 0'a bağlarsan çizim başarısız olur. Doğru sıra: önce VAO'yu çöz, sonra istersen EBO'yu.

17.2 Uniform'lar: CPU'dan Shader'a Veri

Öznitelikler her köşe için farklıdır; **uniform**'lar ise tüm çizim boyunca sabit, CPU'dan gönderilen global değerlerdir. Örnek: rengi zamanla değiştiren bir üçgen.

```

1 // FRAGMENT SHADER
2 #version 330 core
3 out vec4 FragColor;
4 uniform vec4 dynamicColor; // will come from the CPU
5
6 void main() {
7     FragColor = dynamicColor;
8 }

```

```

1 // C++ main loop: oscillate the color over time
2 glUseProgram(shaderProgram); // use the program FIRST!
3 float t = (float)glfwGetTime();

```

```

4 float green = (sin(t) / 2.0f) + 0.5f;    // oscillates in 0..1
5
6 int location = glGetUniformLocation(shaderProgram, "dynamicColor");
7 glUniform4f(location, 0.0f, green, 0.0f, 1.0f); // update the uniform
8
9 glBindVertexArray(VAO);
10 glDrawArrays(GL_TRIANGLES, 0, 3);

```

Dikkat

`glUniform*` çağrısı, güncellenecek uniform'un ait olduğu programın *o an kullanımda* olmasını gerektirir. Yani önce `glUseProgram`, sonra `glUniform`. `glGetUniformLocation` `-1` dönerse uniform bulunamamış (veya kullanılmadığı için silinmiş) demektir.

18. Dokular (Textures)

Doku, bir yüzeye "yapıştırdığımız" resimdir. Her köşeye bir *doku koordinatı* (UV, $[0, 1]$ aralığında) veririz; rasterizasyon bunları interpolasyonla yayar ve fragment shader dokudan örnekleyerek (sample) rengi okur. Resmi yüklemek için hafif `stb_image.h` kütüphanesi standarttır.

```

1 #define STB_IMAGE_IMPLEMENTATION
2 #include "stb_image.h"
3
4 unsigned int texture;
5 glGenTextures(1, &texture);
6 glBindTexture(GL_TEXTURE_2D, texture);
7
8 // Wrapping and filtering settings
9 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
10 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
11 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
12 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
13
14 // Load the image
15 int width, height, channels;
16 stbi_set_flip_vertically_on_load(true); // OpenGL's UV origin is bottom-left!
17 unsigned char* data = stbi_load("wall.jpg", &width, &height, &channels, 0);
18 if (data) {
19     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0,
20                 GL_RGB, GL_UNSIGNED_BYTE, data);
21     glGenerateMipmap(GL_TEXTURE_2D);
22 } else {
23     std::cerr << "Failed to load texture\n";
24 }
25 stbi_image_free(data); // free the CPU-side data (already copied to the GPU)

```

```

1 // VERTEX SHADER -- forward the UV coordinate
2 #version 330 core
3 layout (location = 0) in vec3 aPos;
4 layout (location = 1) in vec2 aUV;
5 out vec2 uv;
6 void main() {
7     gl_Position = vec4(aPos, 1.0);
8     uv = aUV;
9 }

```

```

1 // FRAGMENT SHADER -- sample from the texture
2 #version 330 core
3 in vec2 uv;
4 out vec4 FragColor;
5 uniform sampler2D image; // bound texture unit
6 void main() {
7     FragColor = texture(image, uv); // read the color at uv
8 }

```

18.1 İki Dokuyu Karıştırmak (Texture Units)

Birden fazla doku kullanırken her birini bir *birime* bağlarsın. Klasik alıştırma: iki dokuyu mix ile harmanlamak.

```

1 // Bind two textures to units 0 and 1
2 glActiveTexture(GL_TEXTURE0);
3 glBindTexture(GL_TEXTURE_2D, texture0);
4 glActiveTexture(GL_TEXTURE1);
5 glBindTexture(GL_TEXTURE_2D, texture1);
6
7 // Tell each sampler which unit to read from
8 glUseProgram(shaderProgram);
9 glUniform1i(glGetUniformLocation(shaderProgram, "textureA"), 0);
10 glUniform1i(glGetUniformLocation(shaderProgram, "textureB"), 1);

```

```

1 // FRAGMENT SHADER -- blend two textures 80% / 20%
2 #version 330 core
3 in vec2 uv;
4 out vec4 FragColor;
5 uniform sampler2D textureA;
6 uniform sampler2D textureB;
7 void main() {
8     FragColor = mix(texture(textureA, uv), texture(textureB, uv), 0.2);
9 }

```

Dikkat

stbi_set_flip_vertically_on_load(true) genelde şarttır: resim formatları soldan-üstten başlar, OpenGL doku koordinatları ise soldan-alttan. Çevirmezsen dokular baş aşağı görünür.

19. glm ile Dönüşümleri Uygulamak

Kısım 2'nin matrislerini artık gerçek çizime bağlıyoruz. Bir matrisi shader'a uniform olarak gönderip vertex shader'da köşeleri dönüştüreceğiz.

```

1 #include <glm/glm.hpp>
2 #include <glm/gtc/matrix_transform.hpp>
3 #include <glm/gtc/type_ptr.hpp> // for value_ptr
4
5 // Inside the main loop:
6 glm::mat4 transform(1.0f);
7 transform = glm::translate(transform, glm::vec3(0.5f, -0.5f, 0.0f)); // to bottom-right
8 transform = glm::rotate(transform, (float)glfwGetTime(), // spin over time
9                          glm::vec3(0.0f, 0.0f, 1.0f));

```

```

10
11 glUseProgram(shaderProgram);
12 unsigned int loc = glGetUniformLocation(shaderProgram, "transform");
13 // send the mat4: 1 matrix, do not transpose (GL_FALSE), data pointer
14 glUniformMatrix4fv(loc, 1, GL_FALSE, glm::value_ptr(transform));

```

```

1 // VERTEX SHADER
2 #version 330 core
3 layout (location = 0) in vec3 aPos;
4 layout (location = 1) in vec2 aUV;
5 out vec2 uv;
6 uniform mat4 transform;
7 void main() {
8     gl_Position = transform * vec4(aPos, 1.0); // apply the transform
9     uv = aUV;
10 }

```

Dikkat

glUniformMatrix4fv'nin üçüncü parametresi transpose'dur. GLM matrisleri *sütun-öncelikli* (column-major) saklar ve OpenGL de öyle bekler, bu yüzden **GL_FALSE** verilir. GL_TRUE verirken matris transpoze olur ve dönüşümler bozulur. glm::value_ptr ise mat4'ün ham float işaretçisini verir.

20. 3B'ye Geçiş: Küp ve Derinlik Testi

Şimdi düz üçgenden gerçek 3B'ye geçiyoruz. MVP zincirini kuracak, derinlik tamponunu açacak ve dönen bir küp çizeceğiz.

20.1 Derinlik Testini Açmak

3B'de arkadaki nesneler öndekiler tarafından örtülmeli. Bunu **derinlik testi** (z-buffer) sağlar. Açmazsan üçgenler çizim sırasına göre üst üste biner.

```

1 // Once, during setup:
2 glEnable(GL_DEPTH_TEST);
3
4 // Every frame, clear the depth buffer ALONG WITH the color buffer:
5 glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

```

Dikkat

Derinlik testini açtıysan her karede GL_DEPTH_BUFFER_BIT'i de temizlemeyi unutma. Unutursan bir önceki karenin derinlik değerleri kalır ve nesneler rastgele kaybolur/parçalanır. Bu çok yaygın bir 3B hatasıdır.

20.2 MVP ile Dönen Küp

Küp 36 köşeden oluşur (6 yüz $\times$ 2 üçgen $\times$ 3 köşe). Üç matrisi kurup shader'a gönderelim.

```

1 glEnable(GL_DEPTH_TEST);
2
3 // Main loop:
4 glClearColor(0.1f, 0.1f, 0.12f, 1.0f);
5 glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

```

```

6  glUseProgram(shaderProgram);
7
8  // MODEL: spin the cube on its own axis over time
9  glm::mat4 model = glm::rotate(glm::mat4(1.0f),
10                               (float)glfwGetTime() * glm::radians(50.0f),
11                               glm::vec3(0.5f, 1.0f, 0.0f));
12 // VIEW: pull the camera back (+3 on Z, world moves -3)
13 glm::mat4 view = glm::translate(glm::mat4(1.0f), glm::vec3(0.0f, 0.0f, -3.0f));
14 // PROJECTION: perspective
15 glm::mat4 projection = glm::perspective(glm::radians(45.0f),
16                                         800.0f / 600.0f, 0.1f, 100.0f);
17
18 // Send all three
19 glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "model"),
20                    1, GL_FALSE, glm::value_ptr(model));
21 glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "view"),
22                    1, GL_FALSE, glm::value_ptr(view));
23 glUniformMatrix4fv(glGetUniformLocation(shaderProgram, "projection"),
24                    1, GL_FALSE, glm::value_ptr(projection));
25
26 glBindVertexArray(VAO);
27 glDrawArrays(GL_TRIANGLES, 0, 36); // 36 vertices = full cube

```

```

1  // VERTEX SHADER -- the MVP chain
2  #version 330 core
3  layout (location = 0) in vec3 aPos;
4  layout (location = 1) in vec2 aUV;
5  out vec2 uv;
6  uniform mat4 model;
7  uniform mat4 view;
8  uniform mat4 projection;
9  void main() {
10     // Right to left: model first, then view, then projection
11     gl_Position = projection * view * model * vec4(aPos, 1.0);
12     uv = aUV;
13 }

```

Bilgi

İşte Kısım 2'deki $\mathbf{v}_{clip} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \cdot \mathbf{v}$ formülünün canlı hali. Model küpü döndürür, View onu kameranın önüne taşır, Projection perspektif verir. Bu üç matris 3B grafiğin bel kemiğidir.

21. Kamera Sınıfı (FPS Kamera)

Sabit kamera sıkıcı. Şimdi klavye+fare ile gezilebilen bir FPS kamerası yazacağız. Euler açılarını (yaw, pitch) kullanıp bir yön vektörü üreteceğiz ve lookAt ile view matrisini her karede güncelleyeceğiz.

```

1  #include <glm/glm.hpp>
2  #include <glm/gtc/matrix_transform.hpp>
3
4  enum class Direction { Forward, Backward, Left, Right };
5
6  class Camera {
7  public:
8      glm::vec3 position;

```

```

9   glm::vec3 front  = glm::vec3(0.0f, 0.0f, -1.0f);
10  glm::vec3 up     = glm::vec3(0.0f, 1.0f, 0.0f);
11  float yaw       = -90.0f;    // -90: looks toward -Z at start
12  float pitch      = 0.0f;
13  float speed      = 2.5f;
14  float sensitivity = 0.1f;
15
16  explicit Camera(glm::vec3 startPos) : position(startPos) {}
17
18  glm::mat4 viewMatrix() const {
19      return glm::lookAt(position, position + front, up);
20  }
21
22  // Keyboard: WASD (scaled by deltaTime => FPS-independent)
23  void processKeyboard(Direction dir, float deltaTime) {
24      float velocity = speed * deltaTime;
25      glm::vec3 right = glm::normalize(glm::cross(front, up));
26      if (dir == Direction::Forward) position += front * velocity;
27      if (dir == Direction::Backward) position -= front * velocity;
28      if (dir == Direction::Left)    position -= right * velocity;
29      if (dir == Direction::Right)   position += right * velocity;
30  }
31
32  // Mouse: update the look direction
33  void processMouse(float xOffset, float yOffset) {
34      yaw  += xOffset * sensitivity;
35      pitch += yOffset * sensitivity;
36      // Clamp the pitch to avoid flipping over
37      if (pitch > 89.0f) pitch = 89.0f;
38      if (pitch < -89.0f) pitch = -89.0f;
39      updateFront();
40  }
41
42 private:
43  void updateFront() {
44      // Build a direction vector from yaw/pitch (spherical coordinates)
45      glm::vec3 f;
46      f.x = cos(glm::radians(yaw)) * cos(glm::radians(pitch));
47      f.y = sin(glm::radians(pitch));
48      f.z = sin(glm::radians(yaw)) * cos(glm::radians(pitch));
49      front = glm::normalize(f);
50  }
51 };

```

```

1  // Usage -- in the main loop and callbacks:
2  Camera camera(glm::vec3(0.0f, 0.0f, 3.0f));
3
4  void processInput(GLFWwindow* window, float deltaTime) {
5      if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
6          camera.processKeyboard(Direction::Forward, deltaTime);
7      if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
8          camera.processKeyboard(Direction::Backward, deltaTime);
9      if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
10         camera.processKeyboard(Direction::Left, deltaTime);
11      if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
12         camera.processKeyboard(Direction::Right, deltaTime);
13  }
14

```

```

15 // Mouse callback (store lastX/lastY, skip the very first frame):
16 float lastX = 400.0f, lastY = 300.0f;
17 bool firstMouse = true;
18 void cursorPosCallback(GLFWwindow* win, double xpos, double ypos) {
19     if (firstMouse) { lastX = xpos; lastY = ypos; firstMouse = false; }
20     float xOffset = xpos - lastX;
21     float yOffset = lastY - ypos; // reversed: screen Y goes down, camera Y goes up
22     lastX = xpos; lastY = ypos;
23     camera.processMouse(xOffset, yOffset);
24 }
25
26 // In the draw step: get the view matrix from the camera
27 glm::mat4 view = camera.viewMatrix();

```

İpucu

Bu FPS kamera çapraz çarpımı (sağ yönü bulmak için `cross(front, up)`), küresel koordinatları (yaw/pitch → yön) ve `lookAt`'i bir arada kullanır. Kısım 2'deki her araç burada iş başında. Zoom için scroll callback'inde FOV'u değiştirebilirsin.

22. Temel Aydınlatma: Phong Modeli

Işıksız 3B düz görünür. Phong aydınlatma modeli ışığı üç bileşene ayırır ve gerçekçi bir hacim hissi verir. Her bileşen Kısım 2'deki vektör işlemlerine dayanır.

Bileşen

Fikir

Ambient	Ortam ışığı; her yer minimum aydınlık (sabit çarpan).
Diffuse	Yüzeyin ışığa dönüklüğü: $\max(\mathbf{N} \cdot \mathbf{L}, 0)$ (nokta çarpım!).
Specular	Parlak yansıma noktası; bakış ve yansıma yönüne bağlı.

```

1 // FRAGMENT SHADER -- Phong lighting
2 #version 330 core
3 in vec3 Normal; // surface normal (in world space)
4 in vec3 FragPos; // fragment's world position
5 out vec4 FragColor;
6
7 uniform vec3 lightPos;
8 uniform vec3 lightColor;
9 uniform vec3 objectColor;
10 uniform vec3 viewPos;
11
12 void main() {
13     // 1) Ambient -- constant base light
14     float ambientStrength = 0.1;
15     vec3 ambient = ambientStrength * lightColor;
16
17     // 2) Diffuse -- dot product of normal and light direction
18     vec3 N = normalize(Normal);
19     vec3 L = normalize(lightPos - FragPos);
20     float diff = max(dot(N, L), 0.0);
21     vec3 diffuse = diff * lightColor;

```



```

22
23 // 3) Specular -- how much the reflected ray points at the camera
24 float specularStrength = 0.5;
25 vec3 V = normalize(viewPos - FragPos);
26 vec3 R = reflect(-L, N);
27 float spec = pow(max(dot(V, R), 0.0), 32.0); // 32 = shininess exponent
28 vec3 specular = specularStrength * spec * lightColor;
29
30 // Combine
31 vec3 result = (ambient + diffuse + specular) * objectColor;
32 FragColor = vec4(result, 1.0);
33 }

```

```

1 // VERTEX SHADER -- forward the normal and world position
2 #version 330 core
3 layout (location = 0) in vec3 aPos;
4 layout (location = 1) in vec3 aNormal;
5 out vec3 Normal;
6 out vec3 FragPos;
7 uniform mat4 model;
8 uniform mat4 view;
9 uniform mat4 projection;
10 void main() {
11     FragPos = vec3(model * vec4(aPos, 1.0)); // world position
12     // Normal matrix: fixes normals under non-uniform scaling
13     Normal = mat3(transpose(inverse(model))) * aNormal;
14     gl_Position = projection * view * model * vec4(aPos, 1.0);
15 }

```

Dikkat

Normalleri model matrisiyle doğrudan çarpma! Nesne *eşit olmayan* ölçeklenmişse (ör. X'te 2, Y'de 1) normaller yüzeye dik kalmaz. Doğru dönüşüm *normal matrisidir*: `mat3(transpose(inverse(model)))`. Ayrıca ışık ve bakış yönlerinin **normalize** olduğundan emin ol.

Bilgi

Diffuse bileşenindeki $\max(\mathbf{N} \cdot \mathbf{L}, 0)$ tam olarak Kısım 2'de öğrendiğimiz nokta çarpımının $\cos \theta$ anlamıdır: yüzey ışığa ne kadar dik dönükse o kadar aydınlık. Grafik matematiği ve kodun burada tam olarak buluştuğunu görüyorsun.

KISIM 5

Projeler ve İyi Uygulamalar

23. Soyutlanmış Shader Sınıfı

Shader derleme kodunu her seferinde yazmak yorucu. Yeniden kullanılabilir bir sınıf yazalım: dosyadan okur, derler, hata raporlar ve uniform ayarlamayı kolaylaştırır. Bu, gerçek projelerin standart yapısıdır.

```

1  #include <glad/glad.h>
2  #include <glm/glm.hpp>
3  #include <glm/gtc/type_ptr.hpp>
4  #include <string>
5  #include <fstream>
6  #include <sstream>
7  #include <iostream>
8
9  class Shader {
10 public:
11     unsigned int id;
12
13     Shader(const char* vertexPath, const char* fragmentPath) {
14         std::string vertexCode = readFile(vertexPath);
15         std::string fragmentCode = readFile(fragmentPath);
16         unsigned int vs = compile(GL_VERTEX_SHADER, vertexCode.c_str(), "VERTEX");
17         unsigned int fs = compile(GL_FRAGMENT_SHADER, fragmentCode.c_str(), "FRAGMENT");
18
19         id = glCreateProgram();
20         glAttachShader(id, vs);
21         glAttachShader(id, fs);
22         glLinkProgram(id);
23         checkLink(id);
24         glDeleteShader(vs);
25         glDeleteShader(fs);
26     }
27
28     void use() const { glUseProgram(id); }
29
30     // Uniform helpers
31     void setBool (const std::string& name, bool value) const { glUniform1i(loc(name), (int)
value); }
32     void setInt  (const std::string& name, int value) const { glUniform1i(loc(name), value
); }

```

```

33 void setFloat(const std::string& name, float value) const { glUniform1f(loc(name), value
); }
34 void setVec3 (const std::string& name, const glm::vec3& v) const {
35     glUniform3fv(loc(name), 1, glm::value_ptr(v));
36 }
37 void setMat4 (const std::string& name, const glm::mat4& m) const {
38     glUniformMatrix4fv(loc(name), 1, GL_FALSE, glm::value_ptr(m));
39 }
40
41 private:
42     int loc(const std::string& name) const {
43         return glGetUniformLocation(id, name.c_str());
44     }
45
46     static std::string readFile(const char* path) {
47         std::ifstream file(path);
48         if (!file) { std::cerr << "Cannot open file: " << path << "\n"; return ""; }
49         std::stringstream ss; ss << file.rdbuf();
50         return ss.str();
51     }
52
53     static unsigned int compile(GLenum type, const char* src, const char* label) {
54         unsigned int shader = glCreateShader(type);
55         glShaderSource(shader, 1, &src, nullptr);
56         glCompileShader(shader);
57         int success; glGetShaderiv(shader, GL_COMPILE_STATUS, &success);
58         if (!success) {
59             char log[1024]; glGetShaderInfoLog(shader, 1024, nullptr, log);
60             std::cerr << label << " shader error:\n" << log << "\n";
61         }
62         return shader;
63     }
64
65     static void checkLink(unsigned int program) {
66         int success; glGetProgramiv(program, GL_LINK_STATUS, &success);
67         if (!success) {
68             char log[1024]; glGetProgramInfoLog(program, 1024, nullptr, log);
69             std::cerr << "Program link error:\n" << log << "\n";
70         }
71     }
72 };

```

```

1 // See how much this simplifies usage:
2 Shader shader("shaders/basic.vert", "shaders/basic.frag");
3
4 // In the main loop:
5 shader.use();
6 shader.setMat4("model",      model);
7 shader.setMat4("view",      camera.viewMatrix());
8 shader.setMat4("projection", projection);
9 shader.setVec3("lightColor", glm::vec3(1.0f));

```

İpucu

Bu Shader sınıfı hemen her OpenGL projesinde bulunur. Aynı mantıkla bir Mesh (VAO/VBO/EBO'yu saran), bir Texture ve Camera sınıfı yazarak temiz bir mini motor iskeletine ulaşırısın.

24. Tam Program: Dönen Renkli Küp

Şimdiye kadar öğrendiğimiz her parçayı birleştiren, kopyalayıp derlenebilir tam bir program. GLFW penceresi + GLAD + derinlik testi + MVP + dönen küp.

```

1  #include <glad/glad.h>
2  #include <GLFW/glfw3.h>
3  #include <glm/glm.hpp>
4  #include <glm/gtc/matrix_transform.hpp>
5  #include <glm/gtc/type_ptr.hpp>
6  #include <iostream>
7
8  const char* vertexSrc = R"(
9  #version 330 core
10 layout (location = 0) in vec3 aPos;
11 layout (location = 1) in vec3 aColor;
12 out vec3 vColor;
13 uniform mat4 mvp;
14 void main() {
15     gl_Position = mvp * vec4(aPos, 1.0);
16     vColor = aColor;
17 }";
18
19 const char* fragmentSrc = R"(
20 #version 330 core
21 in vec3 vColor;
22 out vec4 FragColor;
23 void main() { FragColor = vec4(vColor, 1.0); }");
24
25 void framebufferSizeCallback(GLFWwindow*, int width, int height) {
26     glViewport(0, 0, width, height);
27 }
28
29 int main() {
30     glfwInit();
31     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
32     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
33     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
34
35     GLFWwindow* window = glfwCreateWindow(800, 600, "Spinning Cube", nullptr, nullptr);
36     glfwMakeContextCurrent(window);
37     glfwSetFramebufferSizeCallback(window, framebufferSizeCallback);
38     gladLoadGLLoader((GLADloadproc)glfwGetProcAddress);
39     glEnable(GL_DEPTH_TEST);
40
41     // 36 vertices, each face a different color
42     float vertices[] = {
43         // position          // color
44         -0.5f,-0.5f,-0.5f,  1,0,0,   0.5f,-0.5f,-0.5f,  1,0,0,   0.5f, 0.5f,-0.5f,  1,0,0,
45         0.5f, 0.5f,-0.5f,  1,0,0,  -0.5f, 0.5f,-0.5f,  1,0,0,  -0.5f,-0.5f,-0.5f,  1,0,0,
46         -0.5f,-0.5f, 0.5f,  0,1,0,   0.5f,-0.5f, 0.5f,  0,1,0,   0.5f, 0.5f, 0.5f,  0,1,0,
47         0.5f, 0.5f, 0.5f,  0,1,0,  -0.5f, 0.5f, 0.5f,  0,1,0,  -0.5f,-0.5f, 0.5f,  0,1,0,
48         -0.5f, 0.5f, 0.5f,  0,0,1,  -0.5f, 0.5f,-0.5f,  0,0,1,  -0.5f,-0.5f,-0.5f,  0,0,1,
49         -0.5f,-0.5f,-0.5f,  0,0,1,  -0.5f,-0.5f, 0.5f,  0,0,1,  -0.5f, 0.5f, 0.5f,  0,0,1,
50         0.5f, 0.5f, 0.5f,  1,1,0,   0.5f, 0.5f,-0.5f,  1,1,0,   0.5f,-0.5f,-0.5f,  1,1,0,
51         0.5f,-0.5f,-0.5f,  1,1,0,   0.5f,-0.5f, 0.5f,  1,1,0,   0.5f, 0.5f, 0.5f,  1,1,0,
52         -0.5f,-0.5f,-0.5f,  0,1,1,   0.5f,-0.5f,-0.5f,  0,1,1,   0.5f,-0.5f, 0.5f,  0,1,1,
53         0.5f,-0.5f, 0.5f,  0,1,1,  -0.5f,-0.5f, 0.5f,  0,1,1,  -0.5f,-0.5f,-0.5f,  0,1,1,

```

```

54     -0.5f, 0.5f,-0.5f, 1,0,1, 0.5f, 0.5f,-0.5f, 1,0,1, 0.5f, 0.5f, 0.5f, 1,0,1,
55     0.5f, 0.5f, 0.5f, 1,0,1, -0.5f, 0.5f, 0.5f, 1,0,1, -0.5f, 0.5f,-0.5f, 1,0,1
56 };
57
58 unsigned int VAO, VBO;
59 glGenVertexArrays(1, &VAO); glGenBuffers(1, &VBO);
60 glBindVertexArray(VAO);
61 glBindBuffer(GL_ARRAY_BUFFER, VBO);
62 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
63 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6*sizeof(float), (void*)0);
64 glEnableVertexAttribArray(0);
65 glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6*sizeof(float),
66     (void*)(3*sizeof(float)));
67 glEnableVertexAttribArray(1);
68
69 // Compile shaders inline (use the Shader class in real projects)
70 auto compile = [](GLenum type, const char* src) {
71     unsigned int s = glCreateShader(type);
72     glShaderSource(s, 1, &src, nullptr); glCompileShader(s); return s;
73 };
74 unsigned int program = glCreateProgram();
75 glAttachShader(program, compile(GL_VERTEX_SHADER, vertexSrc));
76 glAttachShader(program, compile(GL_FRAGMENT_SHADER, fragmentSrc));
77 glLinkProgram(program);
78
79 while (!glfwWindowShouldClose(window)) {
80     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
81         glfwSetWindowShouldClose(window, true);
82
83     glClearColor(0.08f, 0.08f, 0.10f, 1.0f);
84     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
85
86     glm::mat4 model = glm::rotate(glm::mat4(1.0f),
87         (float)glfwGetTime(), glm::vec3(0.5f, 1.0f, 0.0f));
88     glm::mat4 view = glm::translate(glm::mat4(1.0f), glm::vec3(0, 0, -3));
89     glm::mat4 projection = glm::perspective(glm::radians(45.0f),
90         800.0f / 600.0f, 0.1f, 100.0f);
91     glm::mat4 mvp = projection * view * model;
92
93     glUseProgram(program);
94     glUniformMatrix4fv(glGetUniformLocation(program, "mvp"),
95         1, GL_FALSE, glm::value_ptr(mvp));
96     glBindVertexArray(VAO);
97     glDrawArrays(GL_TRIANGLES, 0, 36);
98
99     glfwSwapBuffers(window);
100    glfwPollEvents();
101 }
102
103 glDeleteVertexArrays(1, &VAO);
104 glDeleteBuffers(1, &VBO);
105 glDeleteProgram(program);
106 glfwTerminate();
107 return 0;
108 }

```

İpucu

Bu programı derleyip çalıştırdığında, koyu bir arka planda her yüzü farklı renkte, kendi ekseninde dönen bir küp göreceksin. Buraya kadar öğrendiğin her kavram — pencere, bağlam, VBO/VAO, shader, MVP, derinlik testi — tek bir çalışan örnekte bir araya geliyor.

25. Kaynak Yönetimi ve Sık Yapılan Hatalar

25.1 Kaynakları Temizlemek

OpenGL nesneleri (VAO, VBO, doku, shader) GPU belleğinde durur; program biterken veya nesne gereksizken **elle silinmelidir**.

```
1 glDeleteVertexArrays(1, &VAO);
2 glDeleteBuffers(1, &VBO);
3 glDeleteBuffers(1, &EBO);
4 glDeleteTextures(1, &texture);
5 glDeleteProgram(shaderProgram);
6 glfwDestroyWindow(window);
7 glfwTerminate();
```

25.2 Hata Ayıklama Kontrol Listesi

Belirti	Olası neden
Siyah/boş ekran	Shader derlenmedi; glUseProgram yok; VAO bağlı değil; near=0.
Segfault (başta)	GLAD yüklenmeden GL çağrısı; bağlam güncel değil.
Nesne bozuk/parçalı	Derinlik tamponu temizlenmiyor; derinlik testi kapalı.
Her şey ters/aynalı	Matris transpoze (GL_TRUE) veya yanlış çarpım sırası.
Aşırı hızlı/yavaş hareket	Delta time kullanılmıyor.
Doku baş aşağı	stbi_set_flip_vertically_on_load çağrılmadı.
Aydınlatma yanlış	Normaller/yön vektörleri normalize değil; normal matrisi kullanılmadı.

Bilgi

OpenGL hatalarını yakalamak için glGetError() kullanabilir ya da OpenGL 4.3+'ta glDebugMessageCallback ile otomatik hata mesajları alabilirsin. Geliştirme sırasında bunu açmak, sessiz hataları görünür kılar ve saatlerce süren hata avını dakikalara indirir.

25.3 Nereden Devam Etmeli?

Konu	Ne öğrenirsin
Model yükleme (Assimp)	OBJ/FBX gibi hazır 3B modelleri sahneye almak.
Framebuffer'lar	Ekran-dışı çizim, post-processing (bloom, blur).
Gölge haritaları	Gerçekçi gölgeler (shadow mapping).
Instancing	Binlerce nesneyi tek çağrıda çizmek.
PBR	Fizik tabanlı gerçekçi malzemeler.
Dear ImGui	GLFW/OpenGL üstüne hızlı arayüz (debug paneli).

İpucu

Bu rehber seni grafiğin *ne* olduğundan *nasıl* yapıldığına taşıdı: teori → matematik → pencere → üçgen → dokulu, aydınlatılmış, kameralı 3B sahne. Buradan sonrası pratik: küçük sahneler kur, shader'larla oyna, her hatayı bir öğrenme fırsatı gör. İyi çizimler!